

Functional Programming and Type Systems

A seminar in five parts

Spring Quarter 1990

Contents

Reading Material and Schedule

Graph Reduction

- Combinators and program representation

- The reduction rules

- Lazy evaluation

- Recursion and implementation

Polymorphic Type Checking

- What polymorphism is

- A language and its types

- Inferring types

Polymorphic Type Inference

- Polytypes

- The inference system

Continuation Passing Style

- Tail calls and continuations

- The translation scheme

Outline

Reading Material and Schedule

Graph Reduction

Polymorphic Type Checking

Polymorphic Type Inference

Continuation Passing Style

Reading material

1. G. E. Revesz, *Lambda-calculus, Combinators, and Functional Programming*.
2. D. A. Turner, *A New Implementation Technique for Applicative Languages*.
3. L. Cardelli, *Basic Polymorphic Typechecking*.
4. L. Cardelli and P. Wegner, *On Understanding Types, Data Abstraction and Polymorphism*.
5. P. Hudak, *A Survey of Functional Programming*.

Schedule

1. Lambda calculus (Chapter 2 in [1]).
2. Combinatorial logic (Chapter 3 in [1]).
3. Functional programming: motivation and background ([5]).
4. Implementation of lazy functional languages: graph reduction ([2]).
5. Type checking ([3] and [4]).

Weekly seminar, AS room 160, Wednesdays 12:15–2pm. If you need credit you are expected to read a paper and present it for the others.

Outline

Reading Material and Schedule

Graph Reduction

- Combinators and program representation

- The reduction rules

- Lazy evaluation

- Recursion and implementation

Polymorphic Type Checking

Polymorphic Type Inference

Continuation Passing Style

Graph reduction

- An implementation strategy for lazy functional languages.
- Based on combinatorial logic.
- Takes advantage of sharing (*call-by-need*).

Review of combinators

Optimizations

$$\mathbf{S}fgx = fx(gx)$$

$$\mathbf{K}xy = x$$

$$\mathbf{I}x = x$$

$$\mathbf{B}fgx = f(gx)$$

$$\mathbf{C}fgx = fxg$$

$$\mathbf{S}(\mathbf{K}E_1)(\mathbf{K}E_2) \Rightarrow \mathbf{K}(E_1E_2)$$

$$\mathbf{S}(\mathbf{K}E_1)\mathbf{I} \Rightarrow E_1$$

$$\mathbf{S}(\mathbf{K}E_1)E_2 \Rightarrow \mathbf{B}E_1E_2$$

$$\mathbf{S}E_1(\mathbf{K}E_2) \Rightarrow \mathbf{C}E_1E_2$$

Example

$$(x+1)*(x-1) \text{ where } x=7$$

$$\equiv$$

$$\mathbf{S}(\mathbf{B} * (\mathbf{C} + 1))(\mathbf{C} - 1) 7$$

How did we get that?

$$(x+1)*(x-1) \text{ where } x=7$$

$$\equiv$$

$$(\lambda x. * (+ x 1) (- x 1)) 7$$

Apply the bracket abstraction algorithm to $(\lambda x. * (+ x 1) (- x 1))$:

$$S(S(K*) (S(S(K+) I) (K1))) (S(S(K-) I) (K1))$$

Apply the following optimizations:

$$S(K+) I \quad \Rightarrow \quad +$$

$$S(K-) I \quad \Rightarrow \quad -$$

$$S+(K1) \quad \Rightarrow \quad C+1$$

$$S-(K1) \quad \Rightarrow \quad C-1$$

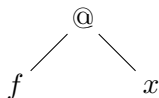
$$S(K*) (C+1) \Rightarrow B*(C+1)$$

and you get $S(B*(C+1)) (C-1)$.

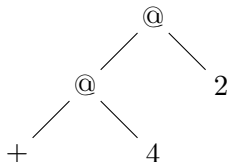
Representation of programs

The combinator expression is stored as a graph.

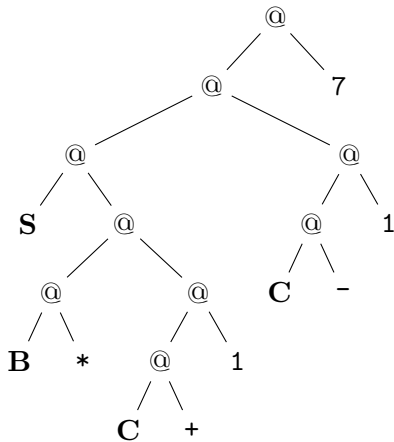
- The application of a function f to an argument x is represented as:



- Functions of several arguments are *curried*:

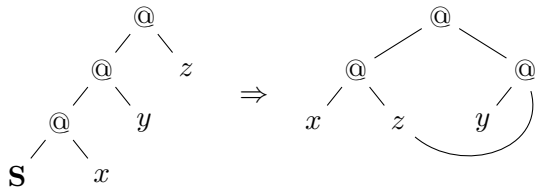


Example

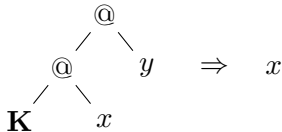


$S(B * (C + 1))(C - 1) 7$

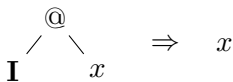
Graph reduction



$$\mathbf{S} \, x \, y \, z = x \, z \, (y \, z)$$

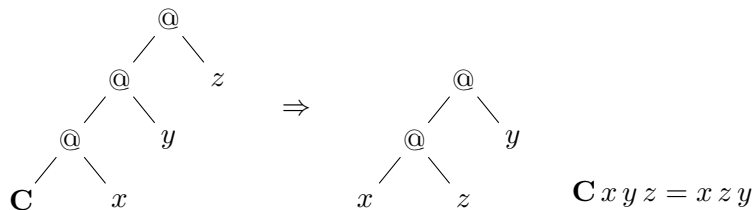
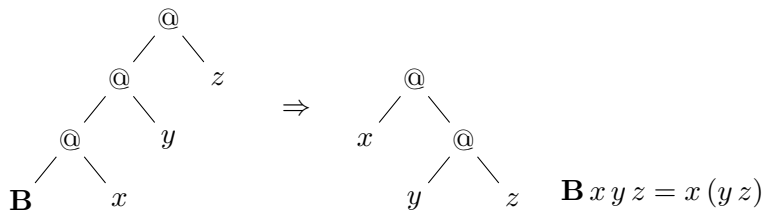


$$\mathbf{K} \, x \, y = x$$



$$\mathbf{I} \, x = x$$

Graph reduction (continued)



Lazy evaluation

What is lazy evaluation?

- Arguments to a function are evaluated when their value is needed and not when the function is applied.
- Arguments should be evaluated at most once and, if possible, not at all.

Lazy evaluation is the natural reduction order for our graphs.

The reduction algorithm

repeat

Find the leftmost node $n \neq @$

if n is an integer **then**

if n is root node **then**

 stop

else

 error

else if $n \in \{\mathbf{S}, \mathbf{K}, \mathbf{I}, \mathbf{B}, \mathbf{C}\}$ **then**

if too few arguments **then**

 stop

else

 apply rewrite rule and

 overwrite the root of redex

 with result

Recursion

Consider a recursive function:

$$fac = \lambda n. n = 0 \rightarrow 1, \ n * fac(n - 1)$$

We use **Y** to get rid of the recursion:

$$fac = \mathbf{Y} \mathbf{H}$$

$$\text{where } \mathbf{H} = \lambda f. \lambda n. n = 0 \rightarrow 1, \ n * f(n - 1)$$

H can be turned into a combinator expression using $\{\mathbf{S}, \mathbf{K}, \mathbf{I}, \mathbf{B}, \mathbf{C}\}$.

But how is **Y** implemented?

Idea: apply the bracket abstraction algorithm to $\lambda h. (\lambda x. hxx)(\lambda x. hxx)$.

This works but there's a better solution.

The Y combinator

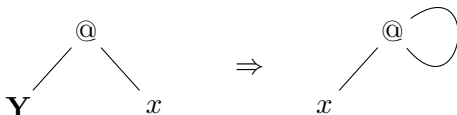
The rule for **Y** is:

$$\mathbf{Y}x = x(\mathbf{Y}x)$$

Applying this again and again:

$$\mathbf{Y}x = x(\mathbf{Y}x) = x(x(\mathbf{Y}x)) = \dots = x(x(x(\dots)))$$

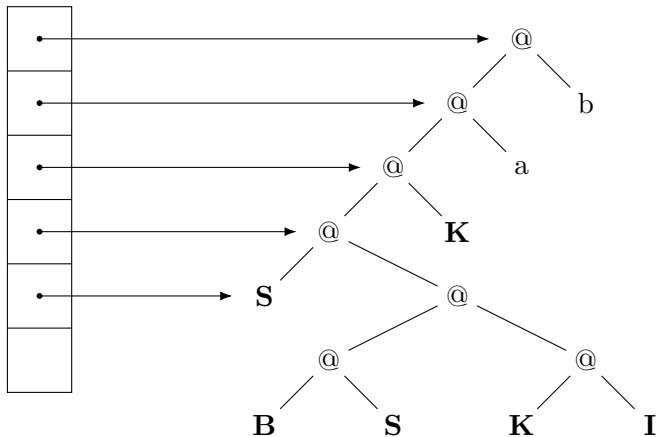
The graph reduction rule for **Y** is:



Now you know why it's called *graph* reduction!

Implementation notes

When we select the next redex we have to remember the arguments. Use a stack while unwinding the spine:



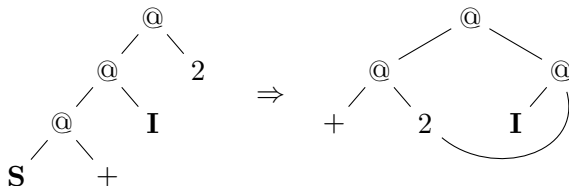
(Another strategy is to reverse the pointers.)

Not all functions are lazy

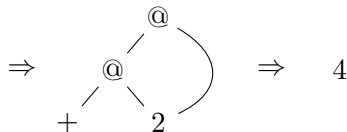
Consider this example: $(\lambda x.x + x)2$.

The function compiles to $\mathbf{S(S(K+)I)I}$, which is equivalent to $\mathbf{S+I}$.

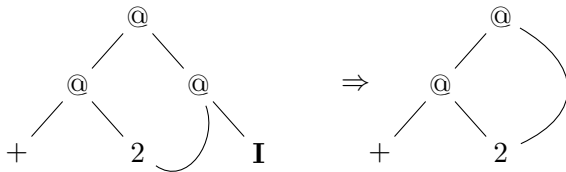
The first reduction step is:



The function $+$ must evaluate its arguments!



Reducing the shared argument



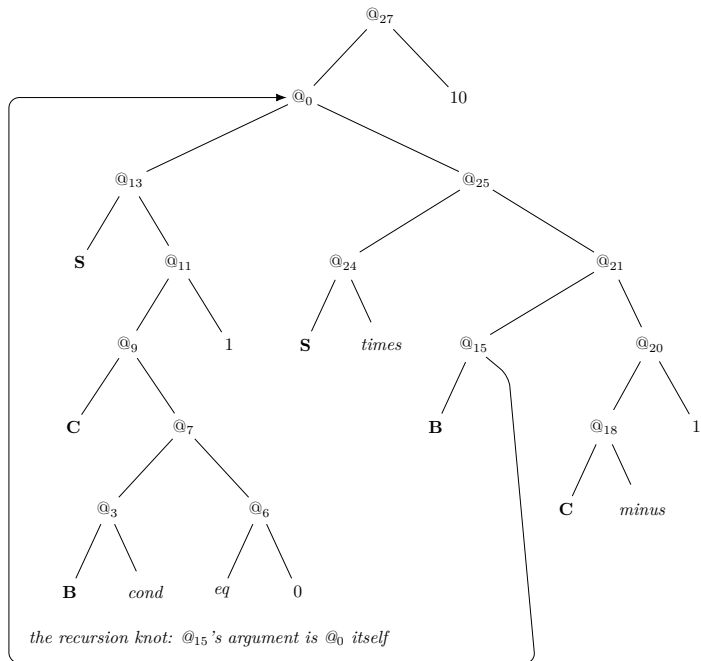
A worked example: *fac* 10

This is what `sk/sk.c` actually builds in the heap for *fac* 10.
The definition sitting at cell `@0` is

$$\mathbf{S} (\mathbf{C} (\mathbf{B} \textit{ cond } (eq\ 0))\ 1) (\mathbf{S} \textit{ times } (\mathbf{B} \langle fac \rangle (\mathbf{C} \textit{ minus } 1)))$$

and $\langle fac \rangle$ is not a copy of that expression but a pointer back to `@0` itself.

Tying that knot is how the implementation gets recursion without ever reducing a **Y** combinator — and it is why the store holds a *graph* rather than a tree.



Outline

Reading Material and Schedule

Graph Reduction

Polymorphic Type Checking

- What polymorphism is

- A language and its types

- Inferring types

Polymorphic Type Inference

Continuation Passing Style

Polymorphic Type Checking

*Luca Cardelli — Basic Polymorphic Typechecking.
Science of Computer Programming 8 (1987) 147–172*

- Infer types in the absence of type declarations
- Static typing eliminates the need for runtime checks
- Polymorphism improves reusability

What is polymorphism?

Languages that associate just one type with each value are called *monomorphic* (e.g. C and Pascal).

This is unnecessarily restrictive; consider counting the number of elements in a list. In Pascal we have to write one function for a list of integers, another for a list of records and so on.

Languages that allow values to have more than one type are called *polymorphic*.

```
- length = fun(l) if null(l) then 0  
                else succ(length(tl(l)));  
> length = fn : 'a list -> int
```

'a is called a *type variable* and can be instantiated to any type.

Explicit and implicit polymorphism

Explicit polymorphism. Type parameters are made explicit in procedure headings. Corresponding type arguments are passed when the procedure is called.

```
let id = fun(t:type) fun(a:t)a
id(int)(3)
id(bool)(true)
```

Implicit polymorphism. Type parameters and type applications have been omitted and must be rediscovered by the compiler. This process is called *type inference*.

```
let id = fun(a)a
id(3)
id(true)
```

A simple applicative language

$$\begin{array}{ll} \textit{Exp} & \rightarrow \textit{Ide} \\ & | \text{ if } \textit{Exp} \text{ then } \textit{Exp} \text{ else } \textit{Exp} \\ & | \text{ fun } (\textit{Ide}) \textit{Exp} \\ & | \text{ let } \textit{Decl} \text{ in } \textit{Exp} \\ & | \textit{Exp} \textit{Exp} \\ & | (\textit{Exp}) \\ \\ \textit{Decl} & \rightarrow \textit{Ide} = \textit{Exp} \\ & | \textit{Decl} \text{ then } \textit{Decl} \\ & | \text{ rec } \textit{Decl} \\ & | (\textit{Decl}) \end{array}$$

```
let rec factorial =  
  fun(n)  
    if zero(n)  
    then succ(0)  
    else times(n)(factorial(pred(n)))
```

Types

Types are defined inductively:

- A type variable $(\alpha, \beta, \dots)$ is a type.
- Nullary type operators are types $(int, bool)$.
- If α and β are types, then so are
 - $\alpha \rightarrow \beta$ (function type)
 - $\alpha \times \beta$ (cartesian product)
 - $\alpha \text{ list}$ (homogeneous list)

Example: $(\alpha \rightarrow bool) \times (\alpha \text{ list}) \rightarrow \alpha \text{ list}$

Types containing type variables are *polymorphic*.

Types not containing type variables are *monomorphic*.

Instantiating type variables

Whenever an occurrence of a type variable is instantiated, all the other occurrences of the same variable must be instantiated to the same value.

Legal instantiations of $\alpha \rightarrow \alpha$:

$$int \rightarrow int$$

$$bool \rightarrow bool$$

$$(\beta \times \gamma) \rightarrow (\beta \times \gamma)$$

An illegal instantiation is $(\beta \times \gamma) \rightarrow int$.

The instantiation process is based on unification. Unification fails when trying to match two different type operators, e.g. *int* and *bool*, or when trying to instantiate a variable to a term containing that variable (like α and $\alpha \rightarrow \beta$).

The latter situation arises in typechecking e.g. $\lambda x.xx$, which is therefore considered illegal.

An initial type environment

<i>true, false</i>	: <i>bool</i>
<i>0, 1, ...</i>	: <i>int</i>
<i>succ, pred</i>	: <i>int</i> $\rightarrow$ <i>int</i>
<i>zero</i>	: <i>int</i> $\rightarrow$ <i>bool</i>
<i>pair</i>	: $\alpha \rightarrow (\beta \rightarrow (\alpha \times \beta))$
<i>fst</i>	: $(\alpha \times \beta) \rightarrow \alpha$
<i>snd</i>	: $(\alpha \times \beta) \rightarrow \beta$
<i>nil</i>	: α <i>list</i>
<i>cons</i>	: $(\alpha \times \alpha$ <i>list</i>) $\rightarrow$ α <i>list</i>
<i>hd</i>	: α <i>list</i> $\rightarrow$ α
<i>tl</i>	: α <i>list</i> $\rightarrow$ α <i>list</i>
<i>null</i>	: α <i>list</i> $\rightarrow$ <i>bool</i>

Example: deducing type constraints

let rec *length* = fun(*l*) if *null*(*l*) then 0 else *succ*(*length*(*tl*(*l*)))

1	<i>null</i>	$\alpha \text{ list} \rightarrow \text{bool}$
2	<i>tl</i>	$\beta \text{ list} \rightarrow \beta \text{ list}$
3	0	<i>int</i>
4	<i>succ</i>	$\text{int} \rightarrow \text{int}$
5	<i>null</i> (<i>l</i>)	<i>bool</i>
6	0	γ
7	<i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	γ
8	if <i>null</i> (<i>l</i>) then 0 else <i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	γ
9	<i>null</i>	$\delta \rightarrow \epsilon$
10	<i>l</i>	δ
11	<i>null</i> (<i>l</i>)	ϵ
12	<i>tl</i>	$\phi \rightarrow \chi$
13	<i>l</i>	ϕ
14	<i>tl</i> (<i>l</i>)	χ
15	<i>length</i>	$\eta \rightarrow \iota$
16	<i>tl</i> (<i>l</i>)	η
17	<i>length</i> (<i>tl</i> (<i>l</i>))	ι
18	<i>succ</i>	$\kappa \rightarrow \lambda$
19	<i>length</i> (<i>tl</i> (<i>l</i>))	κ
20	<i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	λ
21	<i>l</i>	μ
22	if <i>null</i> (<i>l</i>) then 0 else <i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	ν
23	fun (<i>l</i>) if <i>null</i> (<i>l</i>) then 0 else <i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	$\mu \rightarrow \nu$
24	<i>length</i>	π
25	fun (<i>l</i>) if <i>null</i> (<i>l</i>) then 0 else <i>succ</i> (<i>length</i> (<i>tl</i> (<i>l</i>)))	π

Unification

Definition (Substitution). A substitution is a finite set of pairs $\{\alpha_1/t_1, \dots, \alpha_n/t_n\}$ where each α_i is a type variable and each t_i is a type such that $\alpha_i \neq t_i$ and $\alpha_i \neq \alpha_j$ if $i \neq j$. The empty substitution is denoted ϵ .

Definition (Application). Let θ be a substitution $\{\alpha_1/t_1, \dots, \alpha_n/t_n\}$ and E a type. The application $E\theta$ of θ to E is the type obtained by simultaneously replacing t_i for every occurrence of α_i in E . $E\theta$ is called an *instance* of E .

$$\begin{aligned}\alpha &\rightarrow \alpha \{ \alpha / \beta \times \text{int}, \gamma / \beta \times \text{int} \} = \beta \times \text{int} \rightarrow \beta \times \text{int} \\ \beta \times \text{int} &\rightarrow \gamma \{ \alpha / \beta \times \text{int}, \gamma / \beta \times \text{int} \} = \beta \times \text{int} \rightarrow \beta \times \text{int}\end{aligned}$$

Most general unifiers

Definition (Unifier). Let t_1 and t_2 be types and let θ be a substitution. Then θ is called a *unifier* of t_1 and t_2 iff $t_1\theta = t_2\theta$.

Example. $\{\alpha/\beta \times \text{int}, \gamma/\beta \times \text{int}\}$ is a unifier for $t_1 = \alpha \rightarrow \alpha$ and $t_2 = \beta \times \text{int} \rightarrow \gamma$.

Some structures have no unifier (e.g. $\text{int} \rightarrow \alpha$ and bool).

Definition (More general substitution). A substitution θ is *more general* than σ iff there is a substitution ω such that $\sigma = \theta\omega$.

When searching for unifiers interest will be limited to the most general ones — a unifier θ of the types t_1 and t_2 is called a *most general unifier* (*mgu*) iff it is more general than any other unifier of t_1 and t_2 . Moreover, there is a theorem that says that most general unifiers are unique.

Type deduction rules for the typed lambda-calculus

[VAR] $env \vdash x : \text{lookup}(x, env)$

[APP]
$$\frac{env \vdash M : \sigma \rightarrow \tau \quad env \vdash N : \sigma}{env \vdash (MN) : \tau}$$

[ABS]
$$\frac{bind(x, \sigma, env) \vdash M : \tau}{env \vdash (\lambda x:\sigma.M) : \sigma \rightarrow \tau}$$

```
(define (deduce env exp)
  (cond ((isvar? exp)
        (lookup (getvar exp) env))
        ((isapp? exp)
         (let ((t1 (deduce env (getfcn exp)))
               (t2 (deduce env (getarg exp))))
           (if (and (isarrow? t1)
                     (eq? (getdomain t1) t2))
               (getcodomain t1)
               (error "type clash"))))
        ((isabs? exp)
         (let* ((newenv (bind (getparm exp)
                              (getparmtyp exp) env))
                (tau (deduce newenv (getbody exp))))
           (mkarrow (getparmtyp exp) tau)))
        (else (error "expression not recognized"))))
```

Outline

Reading Material and Schedule

Graph Reduction

Polymorphic Type Checking

Polymorphic Type Inference

Polytypes

The inference system

Continuation Passing Style

What is a polymorphic type?

A *polytype* σ is defined as:

$$\begin{aligned}\sigma &::= \tau \mid \forall t. \sigma \\ \tau &::= \textit{int} \mid \textit{bool} \mid t \mid \tau \rightarrow \tau\end{aligned}$$

where t is any type variable.

(A *monotype* τ is a *polytype* without quantifiers.)

Example: $\lambda f. \lambda x. f(fx) : \forall t. (t \rightarrow t) \rightarrow t \rightarrow t$

let $I : \forall t. t \rightarrow t = \lambda x:t. x$ **in** $I_1 I_2$

$I_1: (t \rightarrow t) \rightarrow (t \rightarrow t)$

$I_2: t \rightarrow t$

Because I has a polytype, an occurrence of I can be instantiated independently of other occurrences.

A problem with λ -bound identifiers

Consider the following expression:

$$\lambda f.(f\ 3, f\ true)$$

When typing this expression we know nothing about f 's type. The subexpression $f\ 3$ suggests $f : int \rightarrow t$, but $f\ true$ suggests $f : bool \rightarrow t$. These do not unify.

If we tried to give f a more general type, e.g. $f : t_1 \rightarrow t_2$, we will get an unsound type system because the type of $succ$ matches $t_1 \rightarrow t_2$.

Hence we can't make f polymorphic. We will require λ -bound identifiers to be monomorphic, i.e. all occurrences of the identifier must be instantiated to the same type.

The above example will produce a type error.

Let-bound identifiers are polymorphic

Consider:

$$\text{let } f = \lambda a.a \text{ in } (f_1 \ 3, f_2 \ \text{true})$$

Here we are in a better position, because we know exactly what f is, and the type inference algorithm will infer $f : \forall t.t \rightarrow t$.

Different occurrences of f may be instantiated differently:

$$f_1 : \text{int} \rightarrow \text{int}$$
$$f_2 : \text{bool} \rightarrow \text{bool}$$

Notes on terminology. A *generic* type variable is bound by a quantifier. A *non-generic* type variable is not bound.

The Type Inference System

[IDE] $A.x:\sigma \vdash x:\tau$

If $\sigma = \forall t_1 \dots \forall t_n. \tau'$ then $\tau = [t'_1/t_1] \dots [t'_n/t_n] \tau'$ where $t'_1, \dots, t'_n$ are fresh type variables.

[COND]
$$\frac{A \vdash e : \text{bool} \quad A \vdash e' : \tau \quad A \vdash e'' : \tau}{A \vdash (\text{if } e \text{ then } e' \text{ else } e'') : \tau}$$

[ABS]
$$\frac{A.x:\tau_1 \vdash e : \tau_2}{A \vdash (\lambda x.e) : \tau_1 \rightarrow \tau_2}$$

[COMB]
$$\frac{A \vdash e : \tau_1 \rightarrow \tau_2 \quad A \vdash e' : \tau_1}{A \vdash e(e') : \tau_2}$$

[LETBIND]
$$\frac{A \vdash e' : \tau' \quad A.x:\sigma \vdash e : \tau}{A \vdash (\text{let } x = e' \text{ in } e) : \tau}$$

$\sigma = \forall t_1 \dots \forall t_n. \tau'$ where $\{t_1, \dots, t_n\} = \text{FV}(\tau') - \text{FV}(A)$

[SPEC]
$$\frac{A \vdash e : \forall t. \sigma}{A \vdash e : \sigma[\tau/t]}$$

An Inference Example

Let E be the initial type environment

$E = \{\dots, 5:int, \dots, true:bool, \dots\}$, and write

$E_f = E.f:\forall\alpha.\alpha \rightarrow \alpha$.

$$\mathcal{D}_1:$$

$$\frac{\frac{\frac{}{E.x:\alpha \vdash x:\alpha} [\text{VAR}]}{E \vdash \lambda x.x : \alpha \rightarrow \alpha} [\text{ABS}]}{E \vdash \lambda x.x : \forall\alpha.\alpha \rightarrow \alpha} [\text{GEN}]$$

$$\mathcal{D}_2:$$

$$\frac{\frac{\frac{}{E_f \vdash f:\forall\alpha.\alpha \rightarrow \alpha} [\text{VAR}]}{E_f \vdash f:bool \rightarrow bool} [\text{INST}]}{E_f \vdash f \ true : bool} [\text{APP}] \quad \frac{}{E_f \vdash true:bool} [\text{CON}]}{E_f \vdash (f \ true, f \ 5) : bool \times int} [\text{TUP}]$$

$$\frac{\frac{\frac{}{E_f \vdash f:\forall\alpha.\alpha \rightarrow \alpha} [\text{VAR}]}{E_f \vdash f:int \rightarrow int} [\text{INST}]}{E_f \vdash f \ 5 : int} [\text{APP}] \quad \frac{}{E_f \vdash 5:int} [\text{CON}]}{E_f \vdash (f \ true, f \ 5) : bool \times int} [\text{TUP}]$$

$$\frac{\mathcal{D}_1 \quad \mathcal{D}_2}{E \vdash \mathbf{let} \ f = \lambda x.x \ \mathbf{in} \ (f \ true, f \ 5) : bool \times int} [\text{LET}]$$

Outline

Reading Material and Schedule

Graph Reduction

Polymorphic Type Checking

Polymorphic Type Inference

Continuation Passing Style

- Tail calls and continuations

- The translation scheme

Continuation Passing Style

- A transformation technique used in *Orbit*, a compiler for Scheme.
- Similar to three-address code in conventional compilers.
- Based on lambda-calculus.

Tail-recursion

A tail call is a call which does not push a return address, because no further work remains to be done after the return. The called function will return direct to the caller's caller.

```
(define (fac1 n)
  (if (zero? n) 1
      (* n (fac1 (- n 1)))))
```

fac1 is not tail-recursive; fac-aux is:

```
(define (fac2 n)
  (define (fac-aux n acc)
    (if (zero? n) acc
        (fac-aux (- n 1) (* n acc))))
  (fac-aux n 1))
```

Continuations as a programming style

- All calls are tail-calls.
- Always pass an extra continuation argument K.
- When the called function is finished computing its value it will apply K to that value.

```
(define (fac n K)
  (if (zero? n) (K 1)
      (fac (- n 1)
            (lambda (n1!)
              (K (* n n1!))))))
```

A translation scheme: λ -calculus $\rightarrow$ CPS

Scheme's syntax for λ -calculus:

$$\begin{array}{lll} E & \rightarrow & a \quad (\text{constant, variable}) \\ & | & \lambda x.E \quad (\text{abstraction}) \\ & | & E \ E_1 \ E_2 \ \dots \ E_n \quad (\text{application}) \end{array}$$

The input to the translation scheme $\mathcal{C}$ is a λ -expression and a “raw” continuation K (e.g. *print* or exit to the operating system).

$$\begin{aligned} \mathcal{C}[\![a]\!] K &= K \ a \\ \mathcal{C}[\![\lambda x.E]\!] K &= K(\lambda k.\lambda x.\mathcal{C}[\![E]\!] k) \\ \mathcal{C}[\![E \ E_1 \ \dots \ E_n]\!] K &= \mathcal{C}[\![E_1]\!] (\lambda p.\mathcal{A}[\![E_1 \ \dots \ E_n]\!](p \ k)) \\ \mathcal{A}[\![\]]\! K &= K \\ \mathcal{A}[\![h \ t]\!] K &= \mathcal{C}[\![h]\!] (\lambda k.\mathcal{A}[\![t]\!](K \ k)) \end{aligned}$$

Example

$$\begin{aligned} & \mathcal{C}[(\lambda x.x)3] \text{ print} \\ & \mathcal{C}[(\lambda x.x)] (\lambda p.\mathcal{A}[3](p \text{ print})) \\ & (\lambda p.\mathcal{A}[3](p \text{ print})) (\lambda k.\lambda x.\mathcal{C}[x]k) \\ & (\lambda p.\mathcal{A}[3](p \text{ print})) (\lambda k.\lambda x.k \ x) \\ & (\lambda p.\mathcal{C}[3](p \text{ print})) (\lambda k.\lambda x.k \ x) \\ & (\lambda p.(p \text{ print})3) (\lambda k.\lambda x.k \ x) \end{aligned}$$

Other uses of continuations

Denotational semantics In order to describe jumps in imperative programming languages it is necessary to explicitly pass around the current continuation.

Control mechanism Scheme's interpreter is written in a continuation-based style and it is possible to get hold of the current continuation:

```
(call/cc (lambda (K) <function>))
```

This can be used to implement co-routines, process-abstractions and backtracking.